# Operação Sentinel: Cluster Distribuído Resiliente

Atividade avaliativa em equipe: enunciado e roteiro de implementação (30 pontos)

## 1 Ficha da equipe e do trabalho

**Nome da equipe:** _____

**Líder (quem faz a entrega):** _____

| N. | Nome do integrante | Papel inicial | Papel (troca) |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |

Equipes de 2 a 4 pessoas. Papéis: **Infra** (Compose e Nginx), **Aplicação** (Rust e health checks), **Observabilidade** (watchdog, métricas e alertas), **Relator** (evidências, relatório e README). Em equipes menores, acumulem papéis e troquem a cada duas horas de atividade.

**Caso: Operação Sentinel**

**Cenário.** Uma pequena empresa fictícia publica um serviço web em contêineres. O **Nginx** recebe as requisições e as reparte entre duas instâncias de uma aplicação **Rust**. Um **watchdog** independente verifica a saúde dos nós, e o **Prometheus** coleta métricas. Sua equipe é o time de plataforma: precisa montar o ambiente, definir como saber se o serviço está bom, provocar falhas de propósito e, por fim, diagnosticar um incidente ambíguo.
**Produto final:** repositório (ou ZIP) com configuração, código, evidências, relatório e README, mais uma **apresentação com demonstração ao vivo**.

**Datas importantes**

| | |
|---|---|
| **22/10/2026** | Divulgação do enunciado e início das horas 1 a 5. |
| **05/11/2026** | **Apresentação preliminar** (5 min + 2 min de perguntas, até 5 slides). **2 pontos.** |
| **05 a 12/11/2026** | Horas 6 a 8: experimentos, incidente surpresa (em aula, sem aviso do dia), relatório. |
| **12/11/2026** | **Entrega do pacote final** (itens A a G) e **apresentação final com demonstração ao vivo** (10 min). |

| Etapa | | Etapa | |
|---|---|---|---|
| ☐ | Hora 1 – Briefing e desenho | ☐ | Hora 5 – Watchdog e observabilidade |
| ☐ | Hora 2 – Topologia em contêineres | ☐ | Hora 6 – Experimentos de resiliência |
| ☐ | Hora 3 – Serviço Rust e health checks | ☐ | Hora 7 – Incidente surpresa |
| ☐ | Hora 4 – Nginx e balanceamento | ☐ | Hora 8 – Análise, entrega e demonstração |

Regras: evidência registrada na hora em que acontece; ninguém afirma sem prova; mudanças pequenas, versionadas e reversíveis; testes só dentro do projeto. Avaliação total: 30 pontos.

# 2 Visão geral e limites do laboratório

**Objetivo:** projetar, executar e analisar uma pequena arquitetura web distribuída simulada em contêineres, e comunicar o que foi aprendido.

> **Ideia-chave**
> O que está sendo avaliado são **decisões de engenharia, evidências, diagnóstico e comunicação técnica**. Um sistema que sobe, mas que a equipe não sabe explicar nem diagnosticar, vale pouco.

**Ao final, a equipe será capaz de**

- desenhar uma arquitetura com balanceador, nós de aplicação e monitor, apontando seus **pontos únicos de falha**;
- diferenciar *liveness*, *readiness* e teste sintético, e definir SLIs, métricas e alertas com limiares justificados;
- provocar falhas controladas, **prever** o resultado e comparar com o observado;
- diagnosticar um incidente ambíguo separando **fatos** de **inferências**;
- **apresentar e demonstrar** o sistema funcionando, falhando e se recuperando.

**Disclaimer e limites**

- A rede é simulada pelo Docker Compose em **uma única máquina**. Isso não equivale a várias máquinas físicas, zonas de disponibilidade ou uma rede real com falhas independentes.
- **Nada de carga agressiva** nem ataques contra sistemas externos. Todo teste de falha fica restrito aos contêineres e redes criados para a atividade.
- **Health check verde não prova que a experiência do usuário está boa**, e processo ativo não significa serviço saudável. Métricas e alertas podem atrasar, falhar ou gerar falsos positivos.
- Não exponha portas administrativas nem endpoints de métricas à internet. Não coloque senhas, tokens ou dados pessoais no repositório.
- Se usar imagens externas, registre a origem e **fixe a versão** (por exemplo, `nginx:1.27-alpine`, e não `latest`).
- Não execute comandos destrutivos fora do diretório do projeto.
- **Supervisão:** o professor mantém uma versão de recuperação do projeto e orienta a interrupção de qualquer experimento que afete o computador da turma.

**Entregas obrigatórias**

| | | |
|---|---|---|
| **A** | Diagrama da arquitetura e **mapa dos pontos de falha**. | Hora 1 |
| **B** | `docker-compose.yml` e configurações para subir a topologia. Se não der para implementar tudo, apresente um protótipo executável e **declare as limitações**. | Horas 2 e 3 |
| **C** | Configuração do Nginx **comentada** e descrição do fluxo de uma requisição. | Hora 4 |
| **D** | Plano de monitoramento: **5 métricas, 3 alertas** e a explicação de cada limiar. | Hora 5 |
| **E** | **Matriz de experimentos**: hipótese, procedimento, resultado esperado, evidência e conclusão. | Hora 6 |
| **F** | **Relatório de incidente**: linha do tempo, hipóteses, evidências, decisão e prevenção. | Hora 7 |
| **G** | **README** com comandos de inicialização, teste, falha simulada, recuperação e limpeza. | Todas |
| **H** | **Apresentações**: preliminar (05/11) e final com demonstração (12/11). | 05 e 12/11 |

**Na prática**
**Repositório sugerido:** `docker-compose.yml`, `nginx/`, `rust-app/`, `watchdog/`, `prometheus/`, `docs/` (diagramas, plano, matriz, relatório, slides), `evidencias/` (saídas de comandos e logs) e `README.md`. Faça *commits* pequenos e com mensagens claras.

# 3 Arquitetura-alvo

**Objetivo:** conhecer os componentes, o caminho de uma requisição e os lugares onde o sistema pode quebrar.

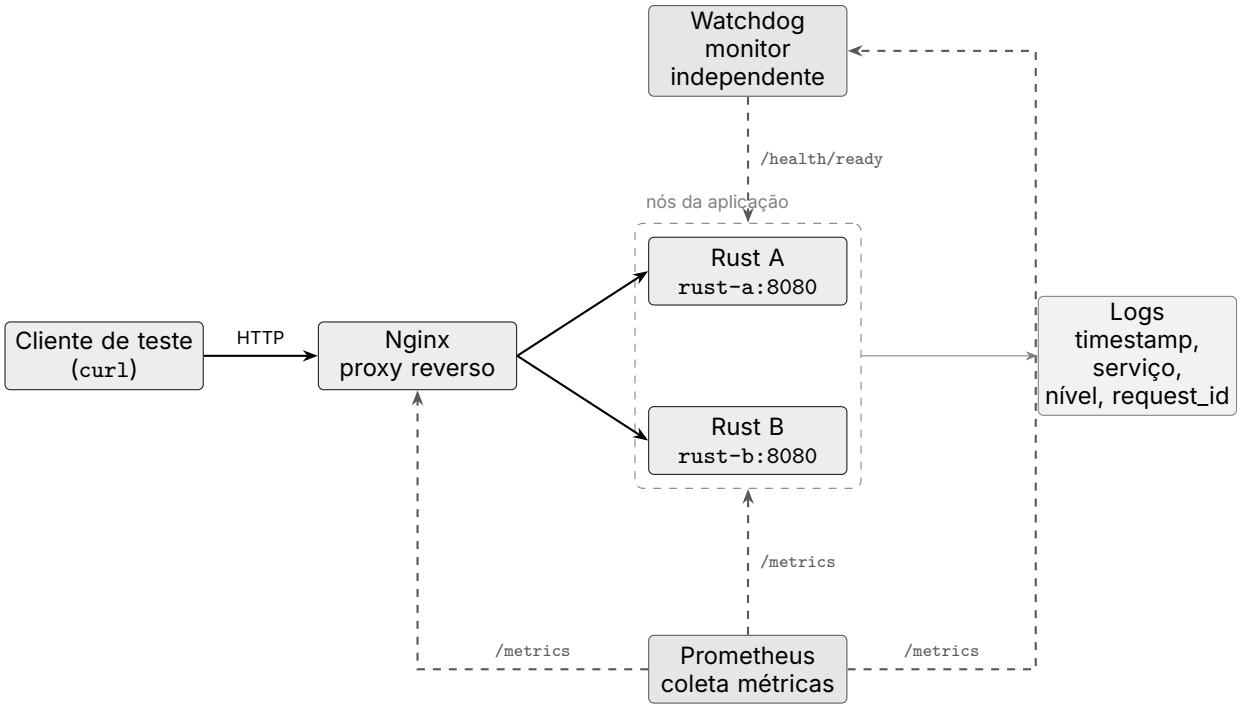



*Figura 1.* Linhas contínuas: caminho do usuário. Tracejadas verdes: observação (sondas e coleta). Todos os serviços escrevem logs, centralizados ou em arquivos locais. O Grafana é opcional.

**Componentes e responsabilidades**

| Componente | Responsabilidade | Endpoint / porta |
|---|---|---|
| Nginx | Recebe o tráfego, distribui entre A e B, aplica timeouts e retry limitado. | `:80` (host: `:8080`) |
| Rust A e B | Atendem as requisições, informam se estão vivos e prontos, expõem métricas. | `/health/live`, `/health/ready`, `/metrics` |
| Watchdog | Sonda os nós, conta falhas consecutivas, registra e decide down/up. | Sonda `/health/ready`; expõe `:9100/metrics` |
| Prometheus | Coleta métricas dos nós Rust, do watchdog e (opcional) do Nginx. | `127.0.0.1:9090` |

**Fluxo de uma requisição (base do item C)**
(1) O cliente envia `GET /` ao Nginx. (2) O Nginx gera um `request_id`, escolhe um nó do *upstream* (revezamento, *round-robin*) e repassa. (3) O nó Rust atende, registra o `request_id` no log e responde. (4) O Nginx devolve a resposta e registra o nó usado, o status e o tempo. (5) Se o nó não responde dentro do timeout, o Nginx tenta **uma vez** o outro nó (retry limitado).

**Falhas a investigar**
Nginx mal configurado; nó Rust fora do ar; processo ativo mas endpoint lento; *readiness* falsa; perda ou atraso entre monitor e nó; métrica ausente; relógios e logs difíceis de correlacionar; retry que amplifica carga; reinício em loop.

# 4 Conceitos essenciais e regras de ouro

**Três perguntas diferentes (não confunda)**

| Verificação | Pergunta | Se falhar... |
|---|---|---|
| **Liveness** | O processo está vivo? | Reiniciar o processo. |
| **Readiness** | Pode receber tráfego agora? | Retirar do balanceamento, *sem* reiniciar. |
| **Teste sintético** | O caminho real do usuário funciona? | Investigar a cadeia inteira (Nginx, nó, dependências). |

**Conceito**
**SLI** (indicador): número que mede a qualidade, como "% de respostas sem erro". **SLO** (objetivo): meta para o SLI, como "99% em 5 minutos". Os quatro sinais clássicos: **disponibilidade, latência, taxa de erro e saturação** (Beyer et al., 2016).

**As 10 regras de ouro**

1. **Defina SLI antes de alerta.** Decida o que é "estar bom" e só então crie alertas.
2. **Separe liveness, readiness e teste sintético**, como na tabela acima.
3. **Monitore de fora** do componente observado sempre que possível, e documente o ponto único de falha do próprio monitor.
4. **Timeouts explícitos e limitados.** Retry só para falhas transitórias, com limite, *backoff* exponencial e *jitter*. Nunca repita indefinidamente operações não idempotentes.
5. **Circuit breaker** reduz chamadas a uma dependência degradada; **bulkhead** limita recursos compartilhados; **balanceamento** distribui tráfego, mas não corrige um bug presente nos dois nós.
6. **Não reinicie automaticamente com base em um único sinal ambíguo.** Exija várias amostras, diferencie falha local de falha de rede e registre a justificativa.
7. **Todo alerta tem** severidade, condição acionável, responsável e procedimento de resposta.
8. **Preserve evidências antes de reiniciar:** logs, métricas, horário, contagem de falhas e estado de cada nó.
9. **Teste a recuperação, não só a falha:** confirme o retorno ao serviço, a drenagem de tráfego e a ausência de reinícios repetidos.
10. **Mudanças pequenas, versionadas e reversíveis.**

> **Por que isso importa?**
> Reiniciar um nó por um único sinal ambíguo pode piorar tudo: se o problema era a *rede entre o monitor e o nó*, o reinício derruba um nó que estava atendendo usuários. Essa é a lição central da Operação Sentinel.

# 5 Cronograma e etapas (8 horas)

Cada etapa tem objetivo, passos e espaço para registrar evidências. **Registre a evidência no momento em que ela acontece** (saída de comando, *print* ou trecho de log com horário).

## 5.1 Hora 1 (00:00–01:00): Briefing e desenho

**Equipe:** _____ **Data:** _____

**Objetivo:** entender o cenário e mapear onde o sistema pode quebrar, antes de escrever qualquer configuração.

**Passos:** (1) Leiam o enunciado e listem componentes e dependências (15 min). (2) Desenhem a arquitetura e marquem em vermelho cada ponto único de falha; dica: o Nginx é um SPOF? E o watchdog? (20 min). (3) Escrevam 3 hipóteses de falha no formato "*Se [evento], então [efeito], porque [mecanismo]*" (25 min).

> **Conceitos**
> **Ponto único de falha (SPOF):** componente cuja queda derruba o serviço. **Dependência:** componente de que outro precisa. **Hipótese de falha:** afirmação testável. **Exemplo:** "Se o Rust A for parado, o Nginx continuará respondendo pelo Rust B, mas as primeiras requisições ao A podem demorar até o timeout, porque o Nginx só retira o nó após falhas."

**Minhas 3 hipóteses de falha (entrega parcial, com o diagrama)**

## 5.2 Hora 2 (01:00–02:00): Topologia em contêineres

**Objetivo:** subir todos os serviços numa rede Compose e saber verificar estado e logs de cada um.

**Passos:** (1) Criem o `docker-compose.yml` com `nginx`, `rust-a`, `rust-b`, `watchdog` e `prometheus` numa rede própria (modelo no Apêndice A). (2) Publiquem no host **apenas** a porta do Nginx; o Prometheus somente em `127.0.0.1`. (3) Documentem no README como verificar estado e logs.

**Conceitos**
**Contêiner:** processo isolado com seu próprio sistema de arquivos. **Docker Compose:** arquivo que descreve vários contêineres e a rede entre eles. **Porta interna** (só dentro da rede) *versus* **porta publicada** (acessível do host).

**Evidência: saída de `docker compose ps` e primeiras linhas de log**

## 5.3 Hora 3 (02:00–03:00): Serviço Rust e health checks

**Equipe:** _____ **Data:** _____

**Objetivo:** ter uma aplicação com liveness, readiness e métricas, e conseguir derrubar sua prontidão de propósito.

**Passos:** (1) Usem ou adaptem o kit (Apêndice C): `/health/live` sempre `200` se o processo responde; `/health/ready` `200` ou `503`; `/metrics` em texto Prometheus. (2) Incluam em cada resposta o nome do nó (`NODE_NAME`) para provar que o balanceamento funciona. (3) Testem um nó saudável e um indisponível: `curl -i http://rust-a:8080/health/ready`.

> **Por que isso importa?**
> Se `/health/live` dependesse do banco e o banco caísse, reiniciar o processo não resolveria nada. Liveness mede só o processo; readiness mede a capacidade de atender.

**Evidência: respostas HTTP (200 e 503) e o comportamento esperado de cada caso**

## 5.4 Hora 4 (03:00–04:00): Nginx e balanceamento

**Objetivo:** configurar o proxy reverso com timeouts, e observar a distribuição e a remoção de um nó.

**Passos:** (1) Configurem o `nginx.conf` (modelo no Apêndice B) e **comentem cada diretiva** (item C). (2) Enviem 20 requisições (`for i in $(seq 20); do curl -s localhost:8080/; done`) e contem quantas cada nó atendeu. (3) Parem um nó e repitam; depois quebrem a configuração de propósito e leiam o erro. (4) Mostrem no log o `request_id`, o nó (`upstream`) e os tempos.

**Conceitos**

**Proxy reverso:** recebe pelo cliente e repassa ao servidor. **Upstream:** grupo de servidores de destino. **Round-robin:** revezamento entre nós. **max_fails / fail_timeout:** quantas falhas retiram o nó do grupo e por quanto tempo. **proxy_next_upstream:** quando tentar outro nó.

**Atenção**

Retry pode **amplificar carga**: se cada requisição falha e é repetida em outro nó, o nó restante recebe o dobro de trabalho justamente quando está sozinho. Por isso o limite de tentativas.

**Contagem de requisições por nó (20 requisições)**

Rust A: _____ Rust B: _____ Com A parado: A _____ B _____

## 5.5 Hora 5 (04:00–05:00): Watchdog e observabilidade

**Equipe:** _____ **Data:** _____

**Objetivo:** medir saúde, latência e falhas consecutivas por um observador independente, e definir métricas e alertas coerentes.

**Passos:** (1) Usem o watchdog do kit ou criem o seu: sonda `/health/ready` a cada 5 s, timeout de 2 s, down após 3 falhas consecutivas. (2) Definam **5 métricas e 3 alertas** (modelo abaixo), com justificativa para cada limiar. (3) Definam **um alerta sobre o próprio monitor**. (4) Confirmem que as métricas **não** estão expostas fora da rede do projeto.

**Conceito**

**Watchdog** ("cão de guarda"): processo que vigia outros. **Falhas consecutivas:** contagem usada para não decidir com uma única amostra. **Saturação:** quão cheio está um recurso (CPU, fila, conexões).

**Item D: alertas (exemplo preenchido e linhas para a equipe)**

| Alerta | Condição e limiar | Justificativa e resposta | Severidade / dono |
|---|---|---|---|
| Nó down | `watchdog_target_down = 1` por 15 s | 3 falhas $\times$ 5 s evitam alarme por perda isolada. Resposta: logs e teste de outro ponto. | Alta / plantão |
| Taxa de erro | 5xx $>$ 2% em 2 min | Abaixo de 2% é ruído; acima, o usuário percebe. Resposta: ver `/erro` e logs. | Alta / plantão |
| Monitor parado | última sondagem há mais de 30 s | 6 intervalos sem sondar. Resposta: salvar logs e só então reiniciar. | Média / equipe |

**Métricas escolhidas (mínimo 5) e o que cada uma mede**

## 5.6 Hora 6 (05:00–06:00): Experimentos de resiliência

**Objetivo:** provocar falhas *uma de cada vez*, prever o resultado antes e comparar com o observado.

**Passos, nesta ordem:** (1) parar o Rust A (`docker compose stop rust-a`); (2) tornar um endpoint lento (`/admin/atraso?ms=2500`, de dentro da rede); (3) interromper o watchdog; (4) simular resposta de erro (`GET /erro`). Registre **previsão, resultado e recuperação**. **Sem carga agressiva.**

**Item E: matriz de experimentos (exemplo preenchido e linhas para a equipe)**

| Hipótese | Procedimento | Esperado | Evidência | Conclusão |
|---|---|---|---|---|
| Parar A não derruba o serviço. | `stop rust-a`; 20 `curl`. | 200 por B; algumas com atraso até 1 s. | Log do Nginx com `upstream=rust-b`; 20 de 20 com 200. | Confirmada; 2 requisições levaram ∼1 s (timeout de conexão). |

## 5.7 Hora 7 (06:00–07:00): Incidente surpresa – Operação Sentinel

**Equipe:** _____ **Data:** _____

**Objetivo:** diagnosticar um incidente ambíguo, sem concluir nada que não tenha evidência.

> **Ideia-chave**
> **O servidor está vivo, mas o sistema não confia nele.** O professor entrega os eventos em etapas. A equipe decide se retira um nó, reinicia algo ou aguarda, e pede os dados que faltam.

### Eventos (versão do aluno)

**09:41:00** O watchdog marca Rust A como down após três verificações sem resposta.

**09:41:10** O Nginx ainda registra respostas 200 de algumas requisições encaminhadas a Rust A.

**09:41:20** A latência global aumentou; Rust B recebe mais tráfego.

**09:41:35** O painel de métricas do watchdog deixa de atualizar, mas não há alerta explícito sobre o próprio monitor.

### Missão

(1) Organizar a **linha do tempo**. (2) Separar **fatos** de **inferências**. (3) Propor **pelo menos três hipóteses**. (4) Escolher a **próxima verificação mais informativa**. (5) Decidir se **retira Rust A do balanceamento** ou **reinicia algo**, com os **riscos** de cada ação. (6) Definir **como confirmar a recuperação**.

**Regra:** nenhuma equipe pode afirmar que um componente *morreu* sem indicar a evidência. Se faltam dados, diga **quais coletaria e como**. **Dicas:** quem diz que o Rust A caiu, o nó ou o observador? Nginx e watchdog veem o mesmo caminho de rede? Qual a idade da última métrica?

### Fatos confirmados

### Hipóteses (mínimo 3)

### Próxima verificação e motivo

### Decisão provisória e riscos

**Como confirmar a recuperação**

### 5.8 Hora 8 (07:00–08:00): Análise, entrega e apresentações

**Objetivo:** fechar os artefatos, revisar os limites da simulação e comunicar os achados com uma demonstração.

**Passos:** (1) Finalizar o relatório (F) e o README (G); alguém de fora da equipe deve conseguir reproduzir tudo só com o README. (2) Revisar os **limites da simulação**: o que este laboratório *não* prova? (3) Ensaiar a apresentação final e a demonstração. (4) Incluir **uma melhoria priorizada** (com justificativa) e **um teste futuro**.

**Item F: relatório de incidente (estrutura)**
(1) Resumo em 3 linhas. (2) **Linha do tempo** com horários. (3) **Fatos** *versus* **inferências**. (4) **Hipóteses** e como cada uma foi testada ou descartada. (5) **Decisão**, riscos e por que é reversível. (6) **Prevenção**: o que mudar na arquitetura, no monitoramento e nos alertas. (7) **Limites** da simulação.

**Apresentação preliminar – 05/11/2026 (2 pontos)**
**Formato:** 5 minutos mais 2 de perguntas, até **5 slides**, todos os integrantes participam. Não é preciso demonstração, mas mostre evidência de que a topologia sobe.
1. Arquitetura e pontos únicos de falha (entrega A).
2. O que já roda: `docker compose ps`, requisição balanceada, health checks (B e C).
3. Métricas e alertas planejados e seus limiares (D).
4. Hipóteses e experimentos previstos (início de E).
5. Riscos, dificuldades e plano até 12/11.

**Avaliação:** clareza, coerência com o que foi construído, honestidade sobre o que falta e qualidade do plano.

**Apresentação final e demonstração – 12/11/2026 (3 pontos)**
**Formato:** 10 minutos por equipe: **3 min** de slides, **5 min** de demonstração ao vivo e **2 min** de perguntas. Conteúdo dos slides: arquitetura final, achado mais importante dos experimentos, decisão no incidente, limites da simulação, uma melhoria priorizada e um teste futuro.

## 6 Roteiro da demonstração ao vivo (5 minutos)

**Equipe:** _____ **Data:** _____

**Objetivo:** mostrar o sistema funcionando, falhando e se recuperando, narrando o que se espera ver *antes* de cada comando.

### Roteiro-base (ajustem comandos e falas à sua equipe)

| N. | Passo | O que mostrar e dizer | Tempo |
|---|---|---|---|
| 1 | Estado inicial | `docker compose ps`: todos de pé. Apontar no diagrama quem é quem. | 30 s |
| 2 | Fluxo normal | 6 requisições; mostrar a alternância entre A e B e o mesmo `request_id` no log do Nginx e do Rust. | 45 s |
| 3 | Falha de um nó | `stop rust-a`. Dizer a previsão. Mostrar que o serviço continua, o log do Nginx e o down no watchdog. | 90 s |
| 4 | Vivo, mas não pronto | Tornar `/health/ready` `503` ou lento um nó; mostrar que `live` segue `200`. | 60 s |
| 5 | Recuperação | `start rust-a`. Provar o retorno ao serviço, a drenagem do tráfego e que não há reinício em loop. | 45 s |
| 6 | Falha do monitor | Parar o watchdog; mostrar o alerta de "última sondagem antiga". Dizer quem vigia o vigia. | 30 s |

### Checklist antes de subir ao palco

- ☐ Ambiente limpo: `down`, depois `up -d --build` e teste
- ☐ Comandos copiados num arquivo de roteiro
- ☐ **Plano B**: vídeo de tela (até 2 min) ou *prints* com horário
- ☐ Sem senhas ou tokens visíveis na tela
- ☐ Fonte grande no terminal e janelas organizadas
- ☐ Todos os integrantes falam
- ☐ Nenhuma porta administrativa exposta à rede
- ☐ Cronometrada no ensaio: cabe em 5 min

### Na prática

O professor pode pedir, durante a demonstração, que a equipe **reproduza uma falha da sua matriz de experimentos** e explique o resultado. Quem ensaiou a matriz, responde com tranquilidade. Se algo der errado ao vivo, **narre o que viu, formule uma hipótese e diga como testaria**: isso também é avaliado.

### Quem faz o quê na demonstração

Digita os comandos: _____

Narra e explica: _____

Cronometra e monitora logs: _____

Responde perguntas: _____

# 7 Rubrica de avaliação (30 pontos)

**Critérios e pontuação**

| Critério | Pontos |
|---|---|
| Arquitetura e clareza do diagrama (A) | 4,0 |
| Execução reproduzível e configuração documentada (B, C) | 4,0 |
| Health checks, métricas e alertas coerentes (D) | 5,0 |
| Qualidade dos experimentos e das evidências (E) | 4,0 |
| Diagnóstico do incidente e uso de evidências (F) | 5,0 |
| Relatório, README e limitações (F, G) | 3,0 |
| Apresentação preliminar em 05/11 (H) | 2,0 |
| Apresentação final e demonstração ao vivo em 12/11 (H) | 3,0 |
| **Total** | **30,0** |

**Descontos**

Credenciais expostas, testes fora do ambiente autorizado ou afirmações sem evidência podem reduzir a nota, conforme as regras da disciplina. Entrega após 12/11 segue as regras da disciplina.

**O que uma boa resposta no incidente costuma ter**

Reconhece sinais contraditórios; evita reinício impulsivo; testa a conectividade a partir de mais de um ponto; consulta logs do Nginx e do Rust com timestamps; confere a idade da última métrica; aplica uma decisão temporária e **reversível**, baseada no impacto observado. **Não existe uma única resposta correta.**

**Comandos de referência**

| | |
|---|---|
| Subir | `docker compose up -d --build` |
| Ver estado | `docker compose ps` |
| Ver logs | `docker compose logs --tail=100` |
| Seguir logs | `docker compose logs -f nginx rust-a rust-b watchdog` |
| Parar / restaurar um nó | `docker compose stop rust-a` / `docker compose start rust-a` |
| Testar | `curl -i http://localhost:8080/` |
| Limpeza | `docker compose down` |

Os nomes são ilustrativos e devem ser ajustados ao Compose da equipe.

# 8 Glossário

| Termo | Significado |
|---|---|
| **Alerta** | Aviso automático quando uma condição acionável ocorre. Tem severidade, dono e procedimento. |
| **Backoff exponencial** | Espera crescente entre tentativas (1 s, 2 s, 4 s...) para não sobrecarregar o serviço. |
| **Balanceamento** | Distribuição das requisições entre vários nós. |
| **Bulkhead** | "Antepara": isola recursos para que a falha de uma parte não consuma tudo. Vem das divisórias de um navio. |
| **Circuit breaker** | "Disjuntor": interrompe chamadas a uma dependência com problema por um tempo. |
| **Contêiner** | Processo isolado, com seu próprio sistema de arquivos e rede virtual. |
| **Demonstração** | Execução ao vivo e narrada do sistema, com previsão antes de cada passo. |
| **Docker Compose** | Ferramenta que descreve e sobe vários contêineres juntos. |
| **Evidência** | Registro verificável (comando, log, *print* com horário) que sustenta uma afirmação. |
| **Falha transitória** | Falha temporária que pode passar sozinha, como uma perda breve de rede. |
| **Falso positivo / negativo** | Alerta sem problema real / problema real sem alerta. |
| **Health check** | Verificação da saúde de um serviço por um endpoint ou comando. |
| **Idempotente** | Operação que, repetida, tem o mesmo efeito (`GET`, sim; "cobrar cartão", não). |
| **Jitter** | Variação aleatória na espera do retry, para que nem todos tentem juntos. |
| **Latência** | Tempo entre pedir e receber a resposta. Observe percentis (p95), não só a média. |
| **Liveness** | "O processo está vivo?" Falha leva ao reinício. |
| **Log** | Registro de eventos com horário: timestamp, serviço, nível, `request_id`, resultado. |
| **Métrica** | Medida numérica ao longo do tempo (contador, medidor ou histograma). |
| **Nginx** | Servidor web usado aqui como proxy reverso e balanceador. |
| **Observabilidade** | Capacidade de entender o estado interno pelo que o sistema emite (métricas, logs e traços). |
| **Perda de pacotes** | Dados enviados pela rede que não chegam ao destino. |
| **Ponto único de falha** | (SPOF) Componente cuja falha derruba todo o serviço. |
| **Prometheus** | Sistema que coleta e armazena métricas, consultáveis em PromQL. |
| **Proxy reverso** | Servidor que recebe pelos clientes e repassa aos servidores internos. |
| **Raio de impacto** | O que uma falha ou mudança pode afetar. |
| **Readiness** | "Posso receber tráfego?" Falha leva à retirada do balanceamento. |
| **request_id** | Identificador único de uma requisição, repetido em todos os logs para correlacioná-la. |
| **Retry** | Nova tentativa após uma falha. Precisa de limite, backoff e jitter. |
| **Round-robin** | Distribuição em revezamento: A, B, A, B... |
| **Rust** | Linguagem de programação de sistemas, segura em memória. |
| **Saturação** | Quão cheio está um recurso (CPU, memória, conexões, fila). |
| **SLI / SLO** | Indicador de qualidade / meta para esse indicador. |
| **Teste sintético** | Requisição artificial que percorre o caminho real do usuário. |
| **Timeout** | Tempo máximo de espera por uma resposta antes de desistir. |
| **Upstream** | No Nginx, o grupo de servidores para onde o tráfego é repassado. |
| **Watchdog** | "Cão de guarda": processo independente que vigia outros e avisa ou age quando param. |

# A Modelos de partida: Compose e Nginx

## A.1 docker-compose.yml

```
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["8080:80"]            # unica porta publicada no host
    volumes: ["./nginx/nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [rust-a, rust-b]
    networks: [sentinel]
  rust-a:
    build: ./rust-app
    environment: { NODE_NAME: rust-a }
    networks: [sentinel]
  rust-b:
    build: ./rust-app
    environment: { NODE_NAME: rust-b }
    networks: [sentinel]
  watchdog:
    build: ./watchdog
    environment:
      ALVOS: "rust-a=http://rust-a:8080,rust-b=http://rust-b:8080"
    depends_on: [rust-a, rust-b]
    networks: [sentinel]
  prometheus:
    image: prom/prometheus:v2.53.0
    volumes: ["./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["127.0.0.1:9090:9090"]   # somente local
    networks: [sentinel]
networks:
  sentinel: {}
```

## A.2 nginx/nginx.conf

```
events {}
http {
  # Formato de log com request_id, no de destino e tempos (correlacao com os logs do Rust)
  log_format sentinel '$time_iso8601 nginx INFO request_id=$request_id '
                      'status=$status upstream=$upstream_addr '
                      'upstream_status=$upstream_status req_time=$request_time '
                      'upstream_time=$upstream_response_time';
  access_log /dev/stdout sentinel;
  error_log  /dev/stderr warn;
  upstream rust_pool {
    # max_fails/fail_timeout: apos 3 falhas, o no sai do pool por 10 s
    server rust-a:8080 max_fails=3 fail_timeout=10s;
    server rust-b:8080 max_fails=3 fail_timeout=10s;
  }
  server {
    listen 80;
    location / {
      proxy_pass http://rust_pool;
      proxy_set_header X-Request-Id $request_id;
      # Timeouts explicitos e curtos (regra de ouro 4)
      proxy_connect_timeout 1s;
      proxy_read_timeout    3s;
      proxy_send_timeout    3s;
      # Retry limitado, so para falhas de conexao/timeout; GET e idempotente
      proxy_next_upstream error timeout http_502 http_503;
      proxy_next_upstream_tries 2;
    }
    # Nao exponha /admin nem /metrics pelo Nginx
    location /admin/   { return 404; }
    location /metrics  { return 404; }
  }
}
```

# B Kit de partida e referências

O professor fornece um **kit** com a aplicação Rust e o watchdog; vocês podem **usar, adaptar ou substituir**.

**Rotas do nó Rust**

| | |
|---|---|
| `GET /` | Responde "atendido por *nome-do-nó*" e aplica o atraso artificial configurado. |
| `GET /erro` | Responde `500`, para o experimento de erro. |
| `GET /health/live` | `200` se o processo responde. |
| `GET /health/ready` | `200` ou `503`, conforme o estado de prontidão. |
| `GET /metrics` | Contadores e medidores em texto Prometheus. |
| `GET /admin/ready?v=0\|1` | Liga/desliga a prontidão (**somente rede interna**). |
| `GET /admin/atraso?ms=N` | Define latência artificial em ms (**somente rede interna**). |

**Watchdog (Python, só biblioteca padrão)**

Sonda `/health/ready` de cada nó, conta falhas consecutivas, escreve logs com horário e expõe `:9100/metrics`. Variáveis: `ALVOS`, `INTERVALO` (5 s), `TIMEOUT` (2 s), `LIMITE_FALHAS` (3).

**Na prática**

**Desafios extras (opcionais):** adicionar `request_id` aos logs do Rust; criar um teste sintético que percorre o caminho completo; montar um painel no Grafana; implementar um *circuit breaker* simples no cliente de teste.

**Referências**

Beyer, B. et al. (2016). *Site Reliability Engineering*. O'Reilly.
Nygard, M. (2018). *Release It!*, 2. ed. Pragmatic Bookshelf.
Tanenbaum, A.; van Steen, M. (2017). *Distributed Systems*, 3. ed.
Documentação oficial: Nginx (`ngx_http_upstream_module`), Prometheus, Docker Compose, Rust (`axum`).